

Implement Client Script & UI Policy (Incident)

Phase 5 – Project Development

5.1 Purpose

The purpose of this phase is to implement the designed **UI Policies, UI Policy Action, and Client Scripts** in the ServiceNow Incident Management environment.

The development activities convert the approved project design into working ServiceNow configurations.

5.2 Development Activity 1 – Create the UI Policy

Create the UI Policy required for High Impact Incidents.

Configuration

- **Name:** High Impact Control
- **Table:** Incident
- **Active:** Yes
- **Condition:** Impact is **1 – High**
- **Reverse if false:** Enabled

The UI Policy is configured to make **Assignment Group** mandatory when the Impact condition is satisfied.

5.3 Development Activity 2 – Configure the UI Policy Action

Create a UI Policy Action under the **High Impact Control** UI Policy.

Configuration

- **Field:** Urgency
- **Mandatory:** Leave alone
- **Visible:** Leave alone
- **Read-only:** True

When Impact is High, the Urgency field becomes read-only.

5.4 Development Activity 3 – Create the onChange Client Script

Create the Client Script responsible for automatically updating Urgency.

Source Code:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {  
    if (isLoading || newValue === "") {  
        return;  
    }  
  
    if (newValue === '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency has been set to High.');    }  
}
```

Configuration

- **Name:** Auto set urgency for high impact
- **Table:** Incident
- **Type:** onChange
- **Field:** Impact
- **Active:** Yes

Function

When the Impact field changes to **1 – High**, the Client Script:

- Sets Urgency to High.
- Displays an informational message to the user.

5.5 Development Activity 4 – Create the onSubmit Client Script

Create the Client Script responsible for validating Incident submission.

Source Code:

```
function onSubmit() {  
    if (g_form.getValue('impact') === '1' &&
```

```

g_form.getValue('assigned_to') == "") {

    g_form.addErrorMessage('Assigned To is required for High Impact Incidents.');
```

```

    return false;
}

return true;
}
```

Configuration

- **Name:** Prevent save if Assigned To missing
- **Table:** Incident
- **Type:** onSubmit
- **Active:** Yes

Function

The script checks whether:

- Impact is High, and
- Assigned To is empty.

If both conditions are satisfied, the Incident submission is prevented and an error message is displayed.

If a valid Assigned To value is provided, the Incident can be submitted.

5.6 Development Activity 5 – Create the onCellEdit Client Script

Create the Client Script responsible for controlling State changes made through the Incident list.

Source Code:

```

function onCellEdit(sysIDs, table, oldValue, newValue, callback) {

    if (oldValue != newValue) {

        alert('State cannot be changed directly from the Incident list.');
```

```

        callback(false);

        return;
    }
}
```

```
}  
  
callback(true);  
  
}
```

Configuration

- **Name:** Prevent state change via list edit
- **Table:** Incident
- **Type:** onCellEdit
- **Field:** State
- **Active:** Yes

Function

When a user attempts to change the State directly from the Incident list, the script displays a warning and prevents the list-based State change.

Users can update State through the Incident form instead.

5.7 Development Activity 6 – Verify the Configuration

After creating the configurations, verify the following:

- The UI Policy is active.
- The condition is set to **Impact = 1 – High**.
- Reverse if false is enabled.
- Assignment Group is configured as mandatory.
- The Urgency UI Policy Action is configured as read-only.
- The onChange Client Script is associated with Impact.
- The onSubmit Client Script is active.
- The onCellEdit Client Script is associated with State.
- All configuration names are consistent with the project documentation.

5.8 Development Activity 7 – Initial Functional Check

Perform an initial check of the implemented configuration before formal testing.

1. Open an Incident record.
2. Set Impact to **1 – High**.
3. Observe the Assignment Group field.
4. Observe the Urgency field.
5. Verify the automatic Urgency behavior.
6. Check the submission validation.
7. Open the Incident list.
8. Check the State list-edit behavior.
9. Verify that State can still be changed through the Incident form.

Any configuration issue identified during this check should be corrected before the formal testing phase.

5.9 Development Output

The development phase produces the completed ServiceNow configuration consisting of:

- **High Impact Control UI Policy**
- **Urgency UI Policy Action**
- **Auto set urgency for high impact onChange Client Script**
- **Prevent save if Assigned To missing onSubmit Client Script**
- **Prevent state change via list edit onCellEdit Client Script**

The completed configuration is ready for formal testing in **Phase 6 – Project Testing**.